

Basic (your first LCEL chain) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (what a chain can do beyond one `.invoke()` call at a time). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the `Runnable` pattern](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the `Runnable` pattern

Basic Version

You need three pieces, connected in a row: something that builds the prompt, something that calls the model, and something that reads the reply.

A prompt template is just a sentence with a blank in it, like "Answer: {question}". You fill in the blank later, when you actually call it.

The `|` symbol connects the three pieces into one chain. Think of it like a pipe: whatever comes out of the first piece goes straight into the next one.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

This exercise is about the `Runnable` pattern that every LCEL piece shares. A prompt template, a chat model, and an output parser are all `Runnable` objects — each one takes an input and produces an output in a predictable, swappable way.

`ChatPromptTemplate.from_template("Answer: {question}")` builds a template object — it doesn't call anything yet, it just describes the shape of the prompt, with `{question}` as a placeholder. `prompt | ChatOpenAI() | StrOutputParser()` chains three `Runnable`s together using `|`, which LangChain overloads to mean "feed this one's output into the next one's input." The result is itself a single `Runnable`, so `chain.invoke({"question": "..."})` runs all three steps and returns the final parsed answer.

The imports: `ChatPromptTemplate` and `StrOutputParser` live in `langchain_core`, while `ChatOpenAI` lives in `langchain_openai` — this is the package split the Core Concepts section describes. `ChatOpenAI()` reads its API key and default model name from your environment the same way Doc04's raw client did — pass `model=` and `temperature=` explicitly if you want to be sure which model you're using.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

`chain.invoke(...)` runs the chain once, on one input, and waits for the whole result. But a `Runnable` promises more than that — the exact same chain you just built also supports `.batch(...)` (run it on a *list* of inputs, in parallel where the underlying model supports it) and `.stream(...)` (get pieces of the final output as they're produced, the same idea as Doc04's streaming exercise, but now working through every piece of the chain, not just the raw API call).

Try `chain.batch([{"question": "..."}, {"question": "..."}, {"question": "..."}])` — it returns a list of results, one per input, and for many providers it's noticeably faster than calling `.invoke()` three separate times in a loop, since the requests can run concurrently instead of one after another.

The real design question `.batch()` raises immediately: **what happens if one input in the batch fails while the others succeed?** By default, `.batch()` raises on the first failure and you lose every result, even the ones that already finished — which is often not what you want for, say, processing 50 rows of a spreadsheet where 1 bad row shouldn't throw away 49 good answers.

```
# lcel_chain_basics_practice.py
results = chain.batch(
    [{"question": q} for q in questions],
    return_exceptions=True,
)
```

`return_exceptions=True` changes that: each position in the returned list is either a real result or the `Exception` that happened for that one input — nothing else is lost.

Sketch how you'd loop over `results` and separate the successes from the failures before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both build and call the chain exactly once, with `.invoke(...)`. Advanced shows that the same 3-line chain already supports running on many inputs at once with `.batch(...)` — for free, with no extra code — and asks the real question that only comes up once you do: whether one bad input should be allowed to silently destroy every other result in the same batch.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
build a prompt template with one blank called question

build a chain: prompt, then the model, then a plain-text parser, joined with |

run the chain with question set to "What's 5 + 7?"

print the answer that comes back
```

Here is almost the whole thing — just try running it and reading it line by line:

```
# lcel_chain_basics_practice.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template("Answer: {question}")
chain = prompt | ChatOpenAI() | StrOutputParser()
```

Expected output if you run just this: nothing yet — the chain is built but never called. Add `result = chain.invoke({"question": "What's 5 + 7?"})` and `print(result)` below it.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
import ChatPromptTemplate, ChatOpenAI, StrOutputParser

prompt = ChatPromptTemplate.from_template("Answer: {question}")

chain = prompt | ChatOpenAI(model="...", temperature=0) | StrOutputParser()

result = chain.invoke({"question": "What's 5 + 7?"})

print(result)
```

```
# lcel_chain_basics_practice.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template("Answer: {question}")
chain = prompt | ChatOpenAI(model="gpt-4o-mini", temperature=0) | StrOutputParser
```

Notice the input to `.invoke(...)` is a dictionary whose key (`question`) matches the template's placeholder name exactly. If they don't match, the chain fails while building the prompt, before any API call happens — you'll practice that failure directly in this document's Edge cases exercise. Add the `.invoke(...)` call and print line yourself before checking Hint 3.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```
build the chain the same way as Intermediate

questions = a list of 3 question dicts

results = chain.batch(questions, return_exceptions=True)

for each (question, result) pair:
    if result is an Exception: report it as a failure, with the question
    else: print it as a success
```

Turning that into real code — fill in the missing piece yourself:

```
# lcel_chain_basics_practice.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template("Answer: {question}")
chain = prompt | ChatOpenAI(model="gpt-4o-mini", temperature=0) | StrOutputParser

questions = [
    {"question": "What's 5 + 7?"},
    {"question": "What's the capital of Japan?"},
    {"question": "Name one gas giant planet."},
]

results = chain.batch(questions, return_exceptions=True)

# your turn: loop over `questions` and `results` together (zip them),
# and print each one as either a success or a failure, distinguishing
# an Exception result from a real answer
...
```

Expected output: 3 lines, each showing the matching question and its answer — with the loop structured so that if you swapped in a question that somehow caused an error, that one line would clearly say so instead of crashing the whole script.

Fill in the loop yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both call the chain exactly once, with `.invoke(...)`, and print one answer. Advanced calls the *same* chain on 3 inputs at once with `.batch(...)`, and handles the case where one of them might fail without losing the other 2 — turning a chain that only proved itself once into one that's actually ready to process a real batch of real inputs.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)